

RoPE

Resources

<https://www.youtube.com/watch?v=SMBkImDWOyQ> (start with this)

<https://www.youtube.com/watch?v=o29P0Kpobz0>

<https://www.youtube.com/watch?v=Kv90HqY9IZA> (also one of the best)

[Coding LLama](#)

Another implementation (different from the one in this presentation):

<https://nn.labml.ai/transformers/rope/index.html>

What is positional encoding?

Original sentence

YOUR

CAT

IS

A

LOVELY

CAT

Embedding
(vector of size 512)

952.207
5450.840
1853.448
...
1.658
2671.529

171.411
3276.350
9192.819
...
3633.421
8390.473

621.659
1304.051
0.565
...
7679.805
4506.025

776.562
5567.288
58.942
...
2716.194
5119.949

6422.693
6315.080
9358.778
...
2141.081
735.147

171.411
3276.350
9192.819
...
3633.421
8390.473

+

+

+

+

+

+

Position Embedding
(vector of size 512).
Only computed once
and reused for every
sentence during
training and inference.

...
...
...
...
...
...

1664.068
8080.133
2620.399
...
9386.405
3120.159

...
...
...
...
...
...

...
...
...
...
...
...

...
...
...
...
...
...

1281.458
7902.890
912.970
3821.102
1659.217
7018.620

=

=

=

=

=

=

Encoder Input
(vector of size 512)

...
...
...
...
...
...

1835.479
11356.483
11813.218
...
13019.826
11510.632

...
...
...
...
...
...

...
...
...
...
...
...

...
...
...
...
...
...

1452.869
11179.24
10105.789
...
5292.638
15409.093

Rotary Position Embeddings

RoFORMER: ENHANCED TRANSFORMER WITH ROTARY POSITION EMBEDDING

Jianlin Su

Zhuiyi Technology Co., Ltd.
Shenzhen
bojonesu@wezhuiyi.com

Yu Lu

Zhuiyi Technology Co., Ltd.
Shenzhen
julianlu@wezhuiyi.com

Shengfeng Pan

Zhuiyi Technology Co., Ltd.
Shenzhen
nickpan@wezhuiyi.com

Ahmed Murtadha

Zhuiyi Technology Co., Ltd.
Shenzhen
mengjiayi@wezhuiyi.com

Bo Wen

Zhuiyi Technology Co., Ltd.
Shenzhen
brucewen@wezhuiyi.com

Yunfeng Liu

Zhuiyi Technology Co., Ltd.
Shenzhen
glenliu@wezhuiyi.com

What's the difference between the absolute positional encodings and the relative ones?

- Absolute positional encodings are fixed vectors that are added to the embedding of a token to represent its absolute position in the sentence. So, it deals with **one token at a time**. You can think of it as the pair (latitude, longitude) on a map: each point on earth will have a unique pair.
- Relative positional encodings, on the other hand, deals with two tokens at a time and it is involved when we calculate the attention: since the attention mechanism captures the “intensity” of how much two words are related to each other, relative positional encodings tells the attention mechanism the **distance** between the two words involved in it. So, given **two tokens**, we create a vector that represents their distance.
- Relative positional encodings were introduced in the following paper

Self-Attention with Relative Position Representations

Peter Shaw
Google
petershaw@google.com

Jakob Uszkoreit
Google Brain
usz@google.com

Ashish Vaswani
Google Brain
avaswani@google.com

Absolute Positional Encodings

From "Attention is all you need"

$$e_{ij} = \frac{(x_i W^Q)(x_j W^K)^T}{\sqrt{d_z}}$$



imgflip.com
Attention Mechanism = e_{ij}
(dot product)

Relative Positional Encodings

From "Self-Attention with relative positional representations"

A_{ij}^K = distance
between the 2 tokens

$$e_{ij} = \frac{x_i W^Q (x_j W^K + a_{ij}^K)^T}{\sqrt{d_z}}$$



*NSFW: Menage a trois

imgflip.com

Rotary Position Embeddings: the inner product

- The **dot product** used in the attention mechanism is a type of *inner product*, which can be thought of as a generalization of the dot product.
- Can we find an inner product over the two vectors **q** (query) and **k** (key) used in the attention mechanism that only depends on the two vectors and the relative distance of the token they represent?

Under the case of $d = 2$, we consider two-word embedding vectors $\mathbf{x}_q, \mathbf{x}_k$ corresponds to query and key and their position m and n , respectively. According to eq. (1), their position-encoded counterparts are:

$$\begin{aligned}\mathbf{q}_m &= f_q(\mathbf{x}_q, m), \\ \mathbf{k}_n &= f_k(\mathbf{x}_k, n),\end{aligned}\tag{20}$$

where the subscripts of $\mathbf{q}_m$ and $\mathbf{k}_n$ indicate the encoded positions information. Assume that there exists a function g that defines the inner product between vectors produced by $f_{\{q,k\}}$:

$$\mathbf{q}_m^\top \mathbf{k}_n = \langle f_q(\mathbf{x}_m, m), f_k(\mathbf{x}_n, n) \rangle = g(\mathbf{x}_m, \mathbf{x}_n, n - m),\tag{21}$$

(n-m) relative distance
between them

Rotary Position Embeddings: the inner product

- We can define a function **g** like the following that only depends on the two embeddings vector **q** and **k** and their relative distance

$$f_q(\mathbf{x}_m, m) = (W_q \mathbf{x}_m) e^{im\theta}$$

The product here, we convert it into a complex number with this form

$$f_k(\mathbf{x}_n, n) = (W_k \mathbf{x}_n) e^{in\theta}$$

***** = **Conjugate** of the complex number

$$g(\mathbf{x}_m, \mathbf{x}_n, m - n) = \text{Re}[(W_q \mathbf{x}_m)(W_k \mathbf{x}_n)^* e^{i(m-n)\theta}]$$

- Using **Euler's formula**, we can write it into its matrix form.

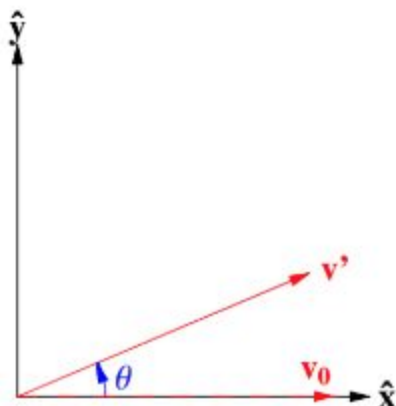
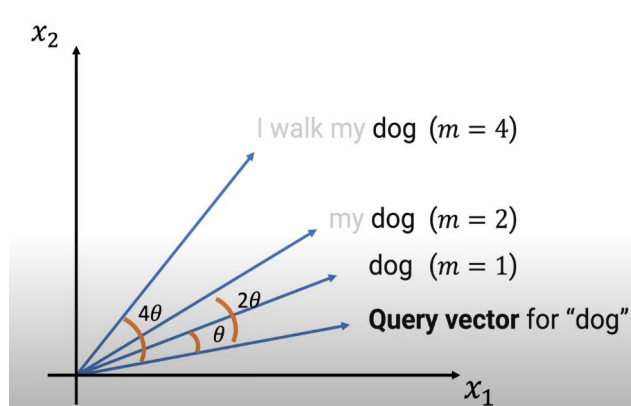
If we imagine as 2D multiplication

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \underbrace{\begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix}}_{\text{Rotation matrix}} \begin{pmatrix} W_{\{q,k\}}^{(11)} & W_{\{q,k\}}^{(12)} \\ W_{\{q,k\}}^{(21)} & W_{\{q,k\}}^{(22)} \end{pmatrix} \begin{pmatrix} x_m^{(1)} \\ x_m^{(2)} \end{pmatrix}$$

m = token position in the sentence

Rotation matrix in a 2d space, hence the name **rotary** position embeddings

Rotary Position Embeddings: the rotation matrix



We multiply v_0 with our R , and the angle will modify

2 tokens that will occupy similar positions, will have similar inclination.

2 tokens with different positions, will have different inclinations

In $\mathbb{R}^2$, consider the matrix that rotates a given vector $\mathbf{v}_0$ by a counterclockwise angle θ in a fixed coordinate system. Then

$$R_\theta = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}, \quad (1)$$

so

$$\mathbf{v}' = R_\theta \mathbf{v}_0. \quad (2)$$

From **Wolfram MathWorld**: <https://mathworld.wolfram.com/RotationMatrix.html>

Rotary Position Embeddings: the general form

Since the matrix is **sparse**, it is not convenient to use it to compute the positional embeddings

This matrix is full of zeros, so we have to simplify the product. (See next slide)

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \mathbf{R}_{\Theta, m}^d \mathbf{W}_{\{q,k\}} \mathbf{x}_m \quad (14)$$

where

$$\mathbf{R}_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix} \quad (15)$$

is the rotary matrix with pre-defined parameters $\Theta = \{\theta_i = 10000^{-2(i-1)/d}, i \in [1, 2, \dots, d/2]\}$. A graphic illustration of RoPE is shown in Figure (1). Applying our RoPE to self-attention in Equation (2), we obtain:

$$\mathbf{q}_m^\top \mathbf{k}_n = (\mathbf{R}_{\Theta, m}^d \mathbf{W}_q \mathbf{x}_m)^\top (\mathbf{R}_{\Theta, n}^d \mathbf{W}_k \mathbf{x}_n) = \mathbf{x}^\top \mathbf{W}_q \mathbf{R}_{\Theta, n-m}^d \mathbf{W}_k \mathbf{x}_n \quad (16)$$

where $\mathbf{R}_{\Theta, n-m}^d = (\mathbf{R}_{\Theta, m}^d)^\top \mathbf{R}_{\Theta, n}^d$. Note that $\mathbf{R}_{\Theta}^d$ is an orthogonal matrix, which ensures stability during the process of encoding position information. In addition, due to the sparsity of $\mathbf{R}_{\Theta}^d$, applying matrix multiplication directly as in Equation (16) is not computationally efficient; we provide another realization in theoretical explanation.

Rotary Position Embeddings: the computational-efficient form

- Given a token with embedding vector $\mathbf{x}$, and the position $\mathbf{m}$ of the token inside the sentence, this is how we compute the position embeddings for the token.

3.4.2 Computational efficient realization of rotary matrix multiplication

Taking the advantage of the sparsity of $R_{\Theta, m}^d$ in Equation (15), a more computational efficient realization of a multiplication of R_{Θ}^d and $\mathbf{x} \in \mathbb{R}^d$ is:

! What we can precompute (to cache it), are the cos and sin matrix. Because as you can see they do not depend of the X matrices. They depend of m and theta

$$R_{\Theta, m}^d \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_{d-1} \\ x_d \end{pmatrix} \otimes \begin{pmatrix} \cos m\theta_1 \\ \cos m\theta_1 \\ \cos m\theta_2 \\ \cos m\theta_2 \\ \vdots \\ \cos m\theta_{d/2} \\ \cos m\theta_{d/2} \end{pmatrix} + \begin{pmatrix} -x_2 \\ x_1 \\ -x_4 \\ x_3 \\ \vdots \\ -x_{d-1} \\ x_d \end{pmatrix} \otimes \begin{pmatrix} \sin m\theta_1 \\ \sin m\theta_1 \\ \sin m\theta_2 \\ \sin m\theta_2 \\ \vdots \\ \sin m\theta_{d/2} \\ \sin m\theta_{d/2} \end{pmatrix} \quad (34)$$

Here we simplified the matrix multiplication from previous slide. Notice the even and the odd numbers (+ and - x)

$$\Theta = \{\theta_i = 10000^{-2(i-1)/d}, i \in [1, 2, \dots, d/2]\}.$$

Let precompute the theta pos frequencies

We compute all $m \cdot \theta$ possible.
For all seq_len

```
def precompute_theta_pos_frequencies(self, seq_len: int, device: str):  
    """  
    Precomputes the RoPE theta position-dependent frequencies.  
  
    Args:  
        seq_len (int): Sequence length.  
        device (str): Device to store tensors.  
  
    Returns:  
        torch.Tensor: Complex exponential frequency tensor.  
    """  
    assert self.head_dim % 2 == 0, "Head dimension must be divisible by 2"
```

$$\theta_i = \frac{1}{\text{base}^{2i/d}}$$

```
# Build the theta parameter  
# According to the formula  $\theta_i = 10000^{-2(i-1)/\text{dim}}$  for  $i = [1, 2, \dots, \text{dim}/2]$   
theta_numerator = torch.arange(0, self.head_dim, 2).float() # Shape: (head_dim / 2,)  
theta = 1.0 / (self.base ** (theta_numerator / self.head_dim)).to(device) # Shape: (head_dim / 2,)  
  
# Construct the positions (the "m" parameter)  
m = torch.arange(seq_len, device=device) # Shape: (seq_len,)  
  
# Multiply each theta by each position using the outer product.  
# Shape: (Seq_Len) outer_product* (Head_Dim / 2) -> (Seq_Len, Head_Dim / 2)  
freqs = torch.outer(m, theta).float() # Shape: (seq_len, head_dim / 2)  
  
# We can compute complex numbers in the polar form  $c = R \cdot \exp(m \cdot \theta)$ , where  $R = 1$  as follows:  
# (Seq_Len, Head_Dim / 2) -> (Seq_Len, Head_Dim / 2)  
freqs_complex = torch.polar(torch.ones_like(freqs), freqs) # Shape: (seq_len, head_dim / 2)  
  
return freqs_complex
```

- `torch.outer(a, b)` computes:

$$\text{freqs}_{m,i} = m \cdot \theta_i$$

- Shape Transformation:

- `positions: (seq_len,)`
- `theta: (head_dim / 2,)`

`outer`, computes all the elements for the first vector (e.g m) with all the elements of the second vector (θ). So all possible combinations

- Resulting Shape: `(seq_len, head_dim / 2)`

4. Compute the Outer Product (`torch.outer`)

python

Copy Edit

```
freqs = torch.outer(m, theta).float()
```

- `torch.outer(a, b)` computes:

$$\text{freqs}_{m,i} = m \cdot \theta_i$$

- Shape Transformation:

- `positions: (seq_len,)`
- `theta: (head_dim / 2,)`
- Resulting Shape: `(seq_len, head_dim / 2)`

5. Convert to Complex Numbers (`torch.polar`)

python

Copy Edit

```
freqs_complex = torch.polar(torch.ones_like(freqs), freqs)
```

- Uses Euler's formula:

$$e^{i\theta} = \cos(\theta) + i\sin(\theta)$$

- Output Shape: `(seq_len, head_dim / 2)`

Generates sequence indices: `[0, 1, 2, ..., seq_len-1]`

$$R_{\Theta, m}^d x = \begin{pmatrix} x_3 \\ x_4 \\ \vdots \\ x_{d-1} \\ x_d \end{pmatrix} \otimes \begin{pmatrix} \cos m\theta_2 \\ \cos m\theta_2 \\ \vdots \\ \cos m\theta_{d/2} \\ \cos m\theta_{d/2} \end{pmatrix} + \begin{pmatrix} -x_4 \\ x_3 \\ \vdots \\ -x_{d-1} \\ x_d \end{pmatrix} \otimes \begin{pmatrix} \sin m\theta_2 \\ \sin m\theta_2 \\ \vdots \\ \sin m\theta_{d/2} \\ \sin m\theta_{d/2} \end{pmatrix}$$

theta

$$\Theta = [\theta_1, \theta_2, \dots, \theta_{d/2}]$$

$$m = [1, 2, \dots, \text{seq-len}]$$

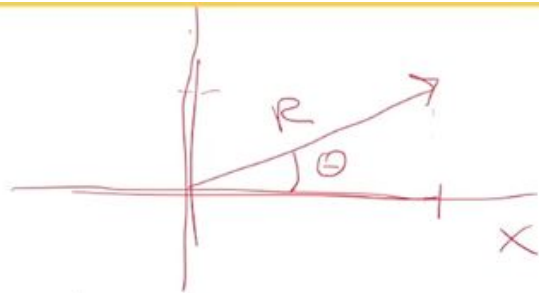
freqs

$$\begin{bmatrix} m_1 \theta_1 & m_1 \theta_2 & m_1 \theta_3 & \dots & m_1 \theta_{d/2} \\ m_2 \theta_1 & m_2 \theta_2 & m_2 \theta_3 & \dots & m_2 \theta_{d/2} \end{bmatrix}$$

Here R=1

$$\begin{bmatrix} \cos(m_1 \theta_1) + i \sin(m_1 \theta_1) & \cos(m_1 \theta_2) + i \sin(m_1 \theta_2) & \dots \end{bmatrix}$$

freqs_complex



$$R e^{i\theta} = R \cos(\theta) + i R \sin(\theta)$$

$$e^{i\theta} = \cos(\theta) + i \sin(\theta)$$

In RoPE, we apply a **position-dependent rotation** to the embeddings. This is done by computing the angles:

$$\theta_{i,m} = m \cdot \frac{1}{\text{base}^{2i/d}}$$

where:

- m represents the **position index** in the sequence.
- i indexes the embedding dimensions (only even indices are used).
- d is the **total embedding dimension**.
- **Base** (typically **10,000**) is used to scale the frequency.

Instead of applying an explicit **2D rotation matrix**, we use **complex numbers** to encode the rotation.

$$R_\theta = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

is equivalent to multiplying by $e^{i\theta}$, where:

$$e^{i(m \cdot \theta)} = \cos(m \cdot \theta) + i \sin(m \cdot \theta)$$

So, **instead of explicitly computing cosine and sine**, we directly compute the complex exponential.

$$e^{i\theta} = \cos(\theta) + i \sin(\theta)$$

In the code:

python

Copy Edit

```
freqs_complex = torch.polar(torch.ones_like(freqs), freqs)
```

- `torch.polar(magnitude, phase)` **creates a complex number** using the given magnitude and angle:

$$c = Re^{i\theta}$$

where:

- `magnitude = 1` (we set $R = 1$ because we only care about rotation, not scaling).
- `phase = m \cdot \theta` (precomputed from `torch.outer`).
- The result is a **complex number in polar form**:

$$e^{i(m\theta)} = \cos(m\theta) + i\sin(m\theta)$$

So instead of computing `cos(m*theta)` and `sin(m*theta)` separately, `torch.polar` does it automatically.

Why Use Complex Multiplication?

- Instead of matrix multiplication, we apply rotation using **complex multiplication**, which is mathematically equivalent but computationally more efficient.
- If we represent a 2D vector as a **complex number**:

$$z = x_{\text{even}} + ix_{\text{odd}}$$

then multiplying by $e^{i\theta}$ **rotates** the vector:

$$z' = z \cdot e^{i\theta}$$

which expands to:

$$x'_{\text{even}} = x_{\text{even}} \cos(\theta) - x_{\text{odd}} \sin(\theta)$$

$$x'_{\text{odd}} = x_{\text{even}} \sin(\theta) + x_{\text{odd}} \cos(\theta)$$

$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \begin{bmatrix} \cos m\theta_2/2 \\ \sin m\theta_2/2 \end{bmatrix} \begin{bmatrix} x_3 \\ x_4 \end{bmatrix} \begin{bmatrix} \cos m\theta_1/2 \\ \sin m\theta_1/2 \end{bmatrix}$

Let say that we have an embedding of 4 dim and we apply a transformation

Then we can convert this as complex numbers

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} \xrightarrow{(1)} \begin{bmatrix} x_1 & x_2 \\ x_3 & x_4 \end{bmatrix} \xrightarrow{(2)} \begin{bmatrix} x_1 + ix_2 \\ x_3 + ix_4 \end{bmatrix}$$

x_complex

`torch.view_as_complex(x_resaped)`

m_1

$\otimes$

$$\begin{bmatrix} \underbrace{\cos(m_1\theta_1)}_{f_1} + i \underbrace{\sin(m_1\theta_1)}_{f_2} \\ \underbrace{\cos(m_1\theta_2)}_{f_3} + i \underbrace{\sin(m_1\theta_2)}_{f_4} \end{bmatrix}$$

freqs_complex

And we multiply with the frequencies matrix (cached) that we calculated before. For simplicity we use f_1, f_2

$$(x_1 + ix_2)(f_1 + if_2) = x_1f_1 + ix_1f_2 + ix_2f_1 - x_2f_2 = x_1f_1 - x_2f_2 + i(x_1f_2 + x_2f_1)$$

So we are doing the multiplication between `x_complex` * `freq_complex`. And then we transform it back here as real values

$$\begin{bmatrix} x_1f_1 - x_2f_2 + i(x_1f_2 + x_2f_1) \\ x_3f_3 - x_4f_4 + i(x_3f_4 + x_4f_3) \end{bmatrix} \xrightarrow{(3)} \begin{bmatrix} x_1f_1 - x_2f_2 & x_1f_2 + x_2f_1 \\ x_3f_3 - x_4f_4 & x_3f_4 + x_4f_3 \end{bmatrix}$$

x_rotated

`torch.view_as_real(x_rotated)`

x_out

m_1

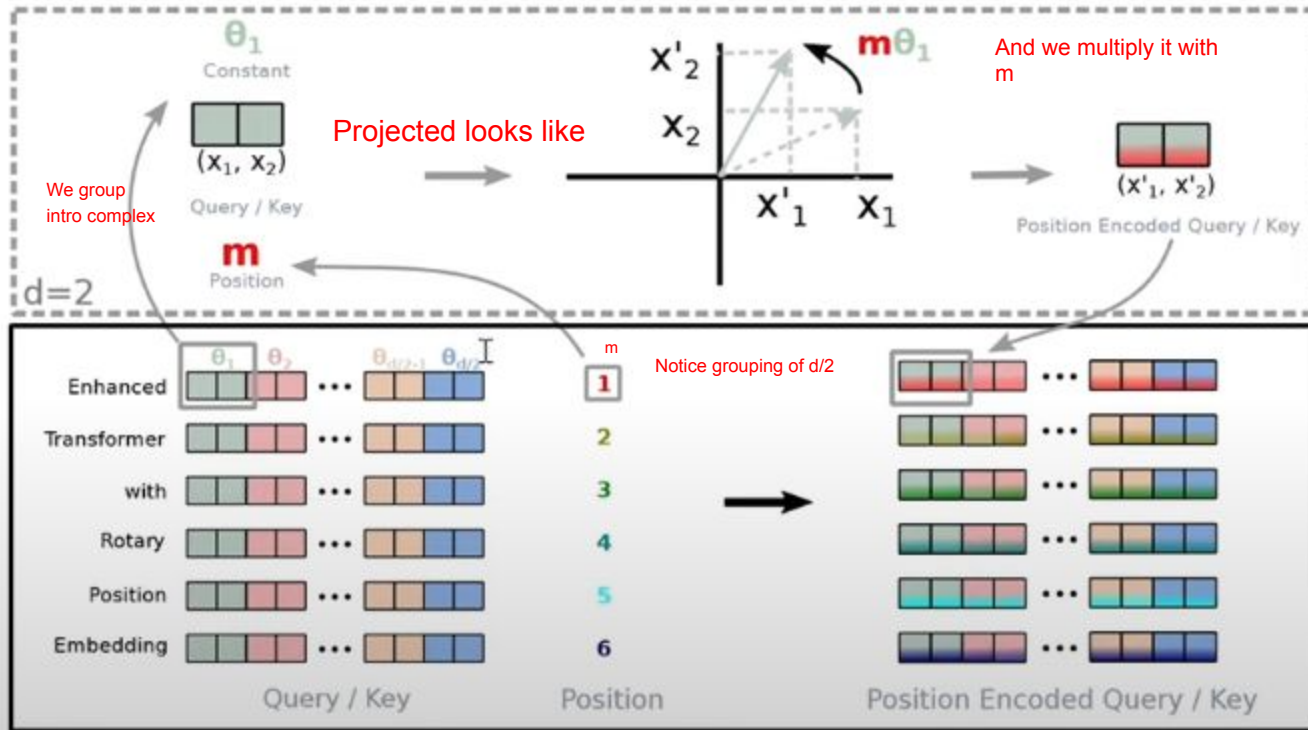
$$(x_1 + ix_2)(f_1 + if_2) = x_1 f_1 + ix_1 f_2 + ix_2 f_1 - x_2 f_2 = x_1 f_1 - x_2 f_2 + i(x_1 f_2 + x_2 f_1)$$

$$\begin{bmatrix} x_1 f_1 - x_2 f_2 + i(x_1 f_2 + x_2 f_1) \\ x_3 f_3 - x_4 f_4 + i(x_3 f_4 + x_4 f_3) \end{bmatrix} \xrightarrow{\textcircled{3}} \begin{bmatrix} [x_1 f_1 - x_2 f_2 & x_1 f_2 + x_2 f_1] \\ [x_3 f_3 - x_4 f_4 & x_3 f_4 + x_4 f_3] \end{bmatrix} \xrightarrow{\textcircled{4}}$$

As you can see now the resulted matrix on the left is what we have on the right. Where f are the sin and cosine

$$\begin{bmatrix} x_1 f_1 - x_2 f_2 \\ x_1 f_2 + x_2 f_1 \\ x_3 f_3 - x_4 f_4 \\ x_3 f_4 + x_4 f_3 \end{bmatrix}$$

$$R_{\Theta, m}^d \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_{d-1} \\ x_d \end{pmatrix} \otimes \begin{pmatrix} \cos m\theta_1 \\ \cos m\theta_1 \\ \cos m\theta_2 \\ \cos m\theta_2 \\ \vdots \\ \cos m\theta_{d/2} \\ \cos m\theta_{d/2} \end{pmatrix} + \begin{pmatrix} -x_2 \\ x_1 \\ -x_4 \\ x_3 \\ \vdots \\ -x_{d-1} \\ x_d \end{pmatrix} \otimes \begin{pmatrix} \sin m\theta_1 \\ \sin m\theta_1 \\ \sin m\theta_2 \\ \sin m\theta_2 \\ \vdots \\ \sin m\theta_{d/2} \\ \sin m\theta_{d/2} \end{pmatrix}$$



As you can see, we can have a word of N dimensions, we take theta of d/2 like in the paper.

- We group each 2 successive theta into 1 Q1 complex number,

Figure 1: Implementation of Rotary Position Embedding(RoPE).

```

def apply_rotary_embeddings(x: torch.Tensor, freqs_complex: torch.Tensor, device: str):
    # Separate the last dimension pairs of two values, representing the real and imaginary parts of the complex number
    # Two consecutive values will become a single complex number
    # (B, Seq_Len, H, Head_Dim) -> (B, Seq_Len, H, Head_Dim/2)
    x_complex = torch.view_as_complex(x.float().reshape(*x.shape[:-1], -1, 2))

    # Reshape the freqs_complex tensor to match the shape of the x_complex tensor. So we need to add the batch dimension and the head dimension
    # (Seq_Len, Head_Dim/2) --> (1, Seq_Len, 1, Head_Dim/2)
    freqs_complex = freqs_complex.unsqueeze(0).unsqueeze(2)

    # Multiply each complex number in the x_complex tensor by the corresponding complex number in the freqs_complex tensor
    # Which results in the rotation of the complex number as shown in the Figure 1 of the paper
    # (B, Seq_Len, H, Head_Dim/2) * (1, Seq_Len, 1, Head_Dim/2) = (B, Seq_Len, H, Head_Dim/2)
    x_rotated = x_complex * freqs_complex

    # Convert the complex number back to the real number
    # (B, Seq_Len, H, Head_Dim/2) -> (B, Seq_Len, H, Head_Dim/2, 2)
    x_out = torch.view_as_real(x_rotated)

    # (B, Seq_Len, H, Head_Dim/2, 2) -> (B, Seq_Len, H, Head_Dim)
    x_out = x_out.reshape(*x.shape)
    return x_out.type_as(x).to(device)

```

```
x_resaped = x.float().reshape(*x.shape[:-1], -1, 2) # (B, Seq_Len, H, Head_Dim/2, 2)
x_complex = torch.view_as_complex(x_resaped) # Convert last two dims to a complex number
```

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} \xrightarrow{\textcircled{1}} \begin{bmatrix} [x_1, x_2] \\ [x_3, x_4] \end{bmatrix} \xrightarrow{\textcircled{2}} \begin{bmatrix} x_1 + i x_2 \\ x_3 + i x_4 \end{bmatrix}$$

```
freqs_complex = freqs_complex.unsqueeze(0).unsqueeze(2) # (1, Seq_Len, 1, Head_Dim/2)
x_rotated = x_complex * freqs_complex # (B, Seq_Len, H, Head_Dim/2)
```

$$\begin{bmatrix} x_1 + i x_2 \\ x_3 + i x_4 \end{bmatrix} \otimes \begin{bmatrix} \underbrace{\cos(m_1 \theta_1)}_{f_1} + i \underbrace{\sin(m_1 \theta_1)}_{f_2} \\ \underbrace{\cos(m_1 \theta_2)}_{f_3} + i \underbrace{\sin(m_1 \theta_2)}_{f_4} \end{bmatrix}$$



$$\begin{bmatrix} x_1 f_1 - x_2 f_2 + i(x_1 f_2 + x_2 f_1) \\ x_3 f_3 - x_4 f_4 + i(x_3 f_4 + x_4 f_3) \end{bmatrix}$$

```

x_rotated = x_complex * freqs_complex # (B, Seq_Len, H, Head_Dim/2)
x_out = torch.view_as_real(x_rotated) # (B, Seq_Len, H, Head_Dim/2, 2)
x_out = x_out.reshape(*x.shape) # Restore original shape

return x_out.type_as(x)

```

freq_complex and then the expression is given here as four terms

$$\begin{bmatrix} x_1 f_1 - x_2 f_2 + i(x_1 f_2 + x_2 f_1) \\ x_3 f_3 - x_4 f_4 + i(x_3 f_4 + x_4 f_3) \end{bmatrix} \xrightarrow{\textcircled{3}} \begin{bmatrix} [x_1 f_1 - x_2 f_2 & x_1 f_2 + x_2 f_1] \\ [x_3 f_3 - x_4 f_4 & x_3 f_4 + x_4 f_3] \end{bmatrix}$$

x_out

As you can see now the resulted matrix on the left is what we have on the right. ¹ Where f are the sin and cosine

$$\begin{bmatrix} x_1 f_1 - x_2 f_2 \\ x_1 f_2 + x_2 f_1 \\ x_3 f_3 - x_4 f_4 \\ x_3 f_4 + x_4 f_3 \end{bmatrix}, \quad \mathbf{R}_{\Theta, m}^d \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_{d-1} \\ x_d \end{pmatrix} \otimes \begin{pmatrix} \cos m\theta_1 \\ \cos m\theta_1 \\ \cos m\theta_2 \\ \cos m\theta_2 \\ \vdots \\ \cos m\theta_{d/2} \\ \cos m\theta_{d/2} \end{pmatrix} + \begin{pmatrix} -x_2 \\ x_1 \\ -x_4 \\ x_3 \\ \vdots \\ -x_{d-1} \\ x_d \end{pmatrix} \otimes \begin{pmatrix} \sin m\theta_1 \\ \sin m\theta_1 \\ \sin m\theta_2 \\ \sin m\theta_2 \\ \vdots \\ \sin m\theta_{d/2} \\ \sin m\theta_{d/2} \end{pmatrix}$$

Rotary Position Embeddings: long-term decay

The authors calculated an **upper bound** for the inner product by varying the distance between two tokens and proved that it decays with the growth of the relative distance.

This means that the “intensity” of relationship between two tokens encoded with Rotary Positional Embeddings will be numerically smaller as the distance between them grows.

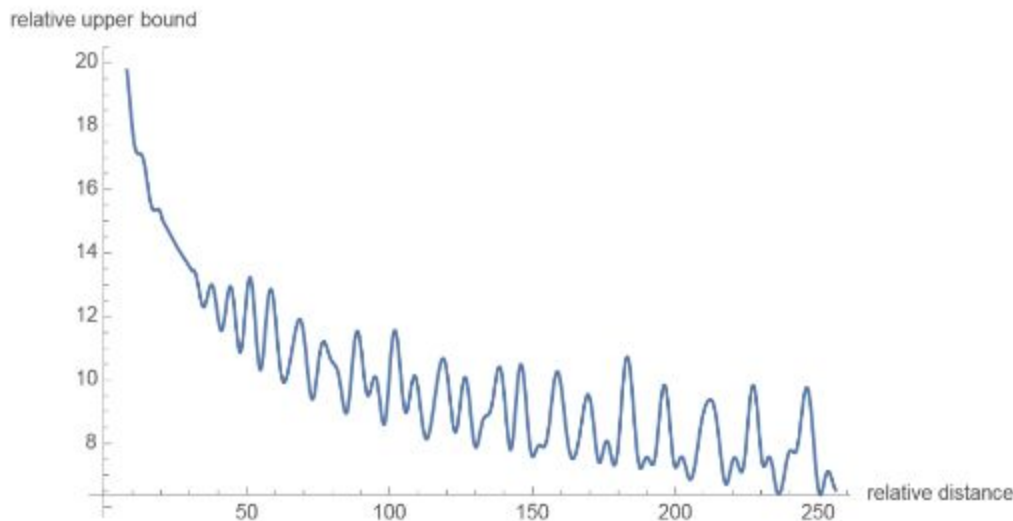


Figure 2: Long-term decay of RoPE.

Rotary Position Embeddings: practical considerations

- The rotary position embeddings are **only applied to the query and the keys**, but not the values.
- The rotary position embeddings are applied after the vector **q** and **k** have been multiplied by the **W** matrix in the attention mechanism, while in the vanilla transformer they're applied before.

```

import torch
import numpy as np

class RotaryPositionEmbedding:
    def __init__(self, dim: int, num_heads: int, base: int = 10000):
        assert dim % num_heads == 0, "Embedding dimension must be divisible by number of heads"
        assert dim % 2 == 0, "Embedding dimension must be even"
        self.dim = dim
        self.num_heads = num_heads
        self.head_dim = dim // num_heads # Dimension per head
        self.base = base

    def precompute_theta_pos_frequencies(self, seq_len: int, device: str):
        assert self.head_dim % 2 == 0, "Head dimension must be divisible by 2"
        # Build the theta parameter
        # According to the formula  $\theta_i = 10000^{(-2(i-1)/dim)}$  for  $i = [1, 2, \dots, dim/2]$ 
        theta_numerator = torch.arange(0, self.head_dim, 2).float() # Shape: (head_dim / 2,)
        theta = 1.0 / (self.base ** (theta_numerator / self.head_dim)).to(device) # Shape: (head_dim / 2,) or dim/2

        # Construct the positions (the "m" parameter)
        m = torch.arange(seq_len, device=device) # Shape: (seq_len,)

        # Multiply each theta by each position using the outer product.
        # Shape: (Seq_Len) outer_product* (Head_Dim / 2) -> (Seq_Len, Head_Dim / 2)
        freqs = torch.outer(m, theta).float() # Shape: (seq_len, head_dim / 2)

        # We can compute complex numbers in the polar form  $c = R * \exp(m * \theta)$ , where  $R = 1$  as follows:
        # (Seq_Len, Head_Dim / 2) -> (Seq_Len, Head_Dim / 2)
        freqs_complex = torch.polar(torch.ones_like(freqs), freqs) # Shape: (seq_len, head_dim / 2)

        return freqs_complex

    def apply_rotary(self, x: torch.Tensor, freqs_complex: torch.Tensor) -> torch.Tensor:
        x_resaped = x.float().reshape(*x.shape[:-1], -1, 2) # (B, Seq_Len, H, Head_Dim/2, 2)
        x_complex = torch.view_as_complex(x_resaped) # Convert last two dims to a complex number
        freqs_complex = freqs_complex.unsqueeze(0).unsqueeze(2) # (1, Seq_Len, 1, Head_Dim/2)
        x_rotated = x_complex * freqs_complex # (B, Seq_Len, H, Head_Dim/2)
        x_out = torch.view_as_real(x_rotated) # (B, Seq_Len, H, Head_Dim/2, 2)
        x_out = x_out.reshape(*x.shape) # Restore original shape

        return x_out.type_as(x)

```

How RoPE is Applied to Multi-Head Attention (MHA)

Now that we have the **Rotary Position Embeddings (RoPE)** implementation, let's discuss **how it integrates with Multi-Head Self-Attention (MHA)** in Transformers.

1. Where Do We Apply RoPE?

In a **standard Multi-Head Attention (MHA)** block, we compute the attention mechanism as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- **Q (Queries), K (Keys), V (Values)** are **linear projections** of the input.
 - **Dot product attention** computes the similarity between **queries and keys**.
 - **Softmax** normalizes the attention scores.
 - The output is a weighted sum of the values.
-

Where Does RoPE Fit?

📌 **RoPE is applied to Queries (Q) and Keys (K), but NOT to Values (V).**

📌 This ensures that **relative positional information** is preserved in the dot product QK^T , without adding explicit position embeddings.

So, before computing attention, we **rotate Q and K** using RoPE.